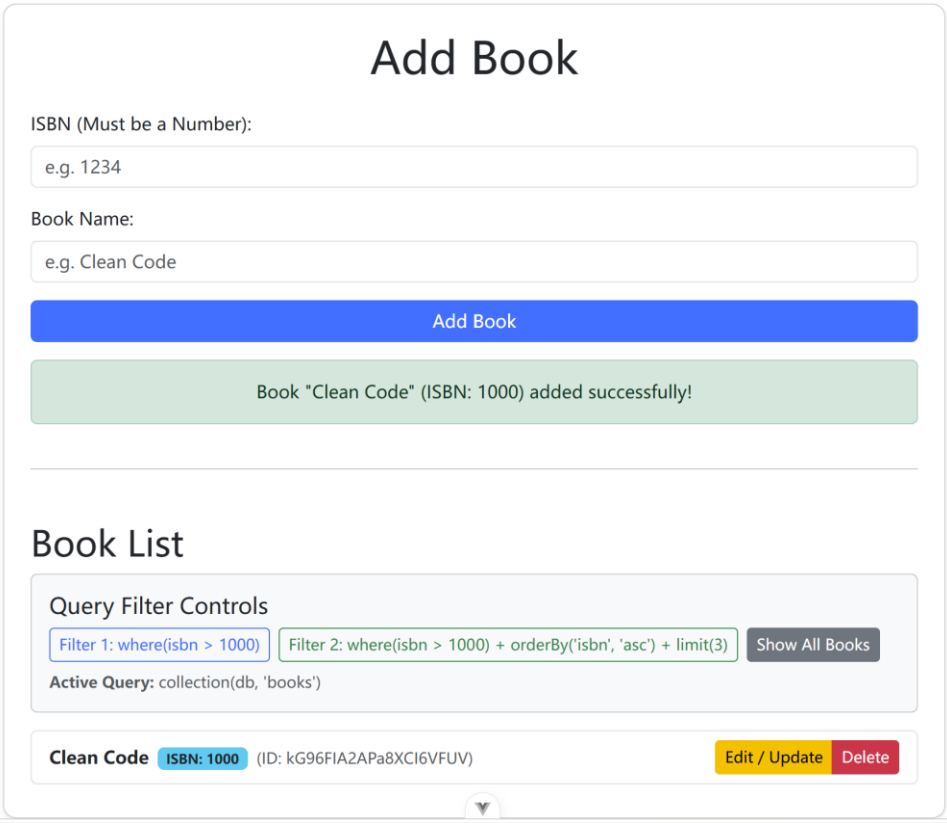
EFOLIO TASK 8.1 (PASS AND CREDIT LEVEL)

WHAT TO SUBMIT (INDIVIDUAL):

Screenshot set 1: Screenshot of the AddBook page on the browser and Visual Studio code



```
const addBook = async () => {
  message.value = ''
  errorMessage.value = ''

  try {
    const isbnNumber = Number(isbn.value)
    if (isNaN(isbnNumber)) {
      errorMessage.value = 'Please enter a valid ISBN number.'
      alert('Please enter a valid ISBN number.')
      return
    }

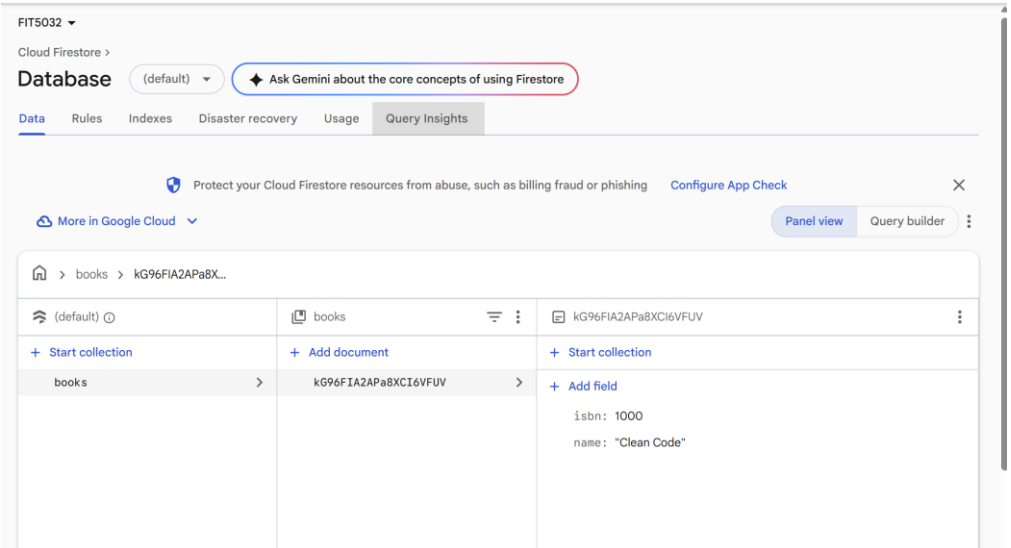
    // Add new document to "books" collection
    const docRef = await addDoc(collection(db, 'books'), {
      isbn: isbnNumber,
      name: name.value
    })

    console.log('Book added with ID: ', docRef.id)
    message.value = `Book "${name.value}" (ISBN: ${isbnNumber}) added successfully!`
    alert('Book added successfully!')

    // Reset inputs
    isbn.value = ''
    name.value = ''

    // Refresh BookList component
    if (bookListRef.value) {
      bookListRef.value.fetchBooksWhere()
    }
  } catch (error: any) {
    console.error('Error adding book: ', error)
    errorMessage.value = `Error adding book: ${error.message || error}`
  }
}
```

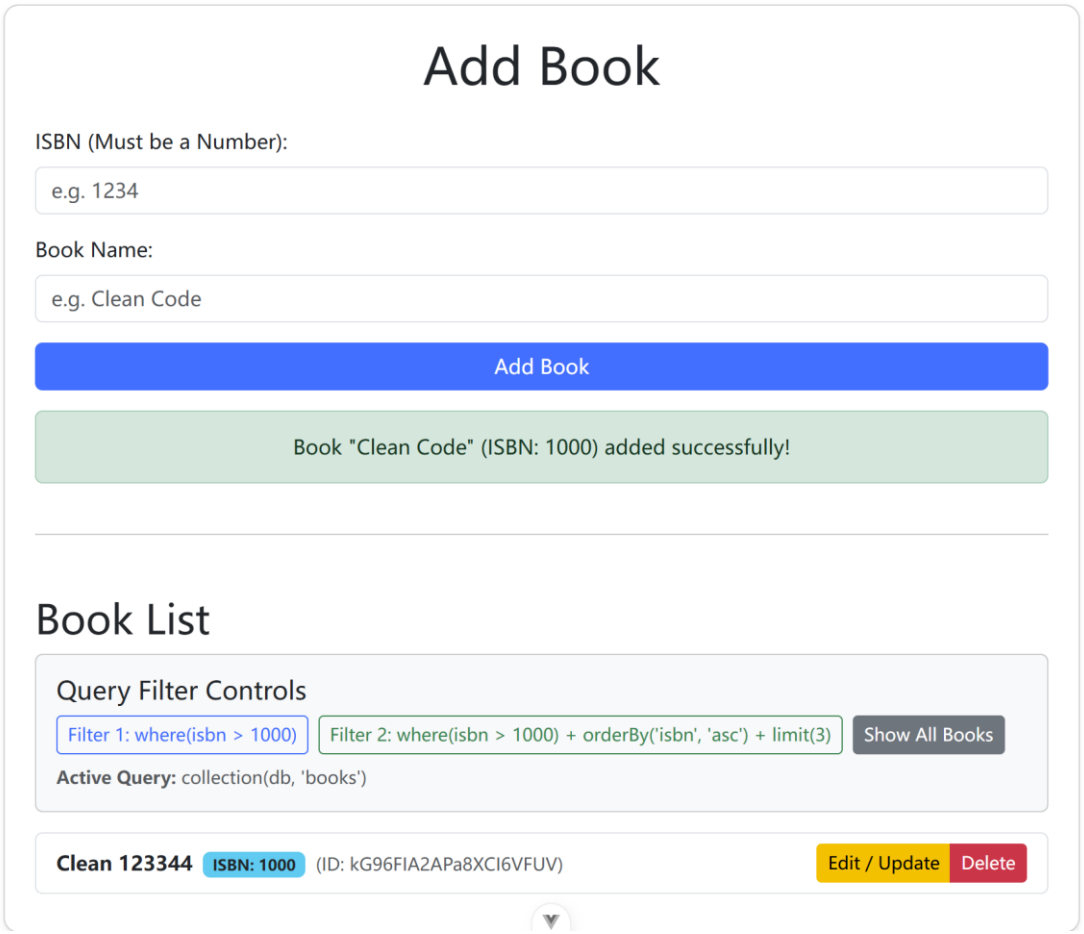
Screenshot set 2: Screenshot of the Firestore page that contains newly added data

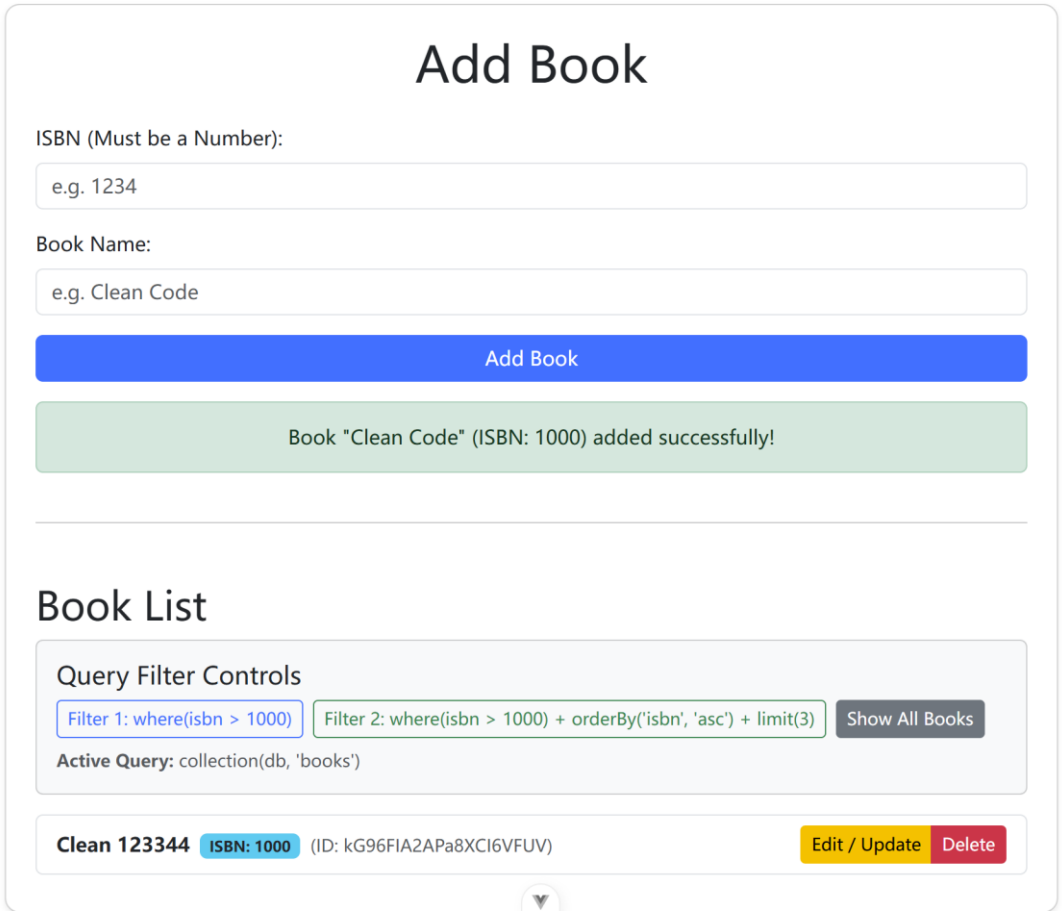


EFOLIO TASK 8.2 (DISTINCTION AND HIGH DISTINCTION LEVEL)

WHAT TO SUBMIT (INDIVIDUAL):

Screenshot set 1: Screenshot of browser and visual studio code that indicates you can update and delete the data from Firestore

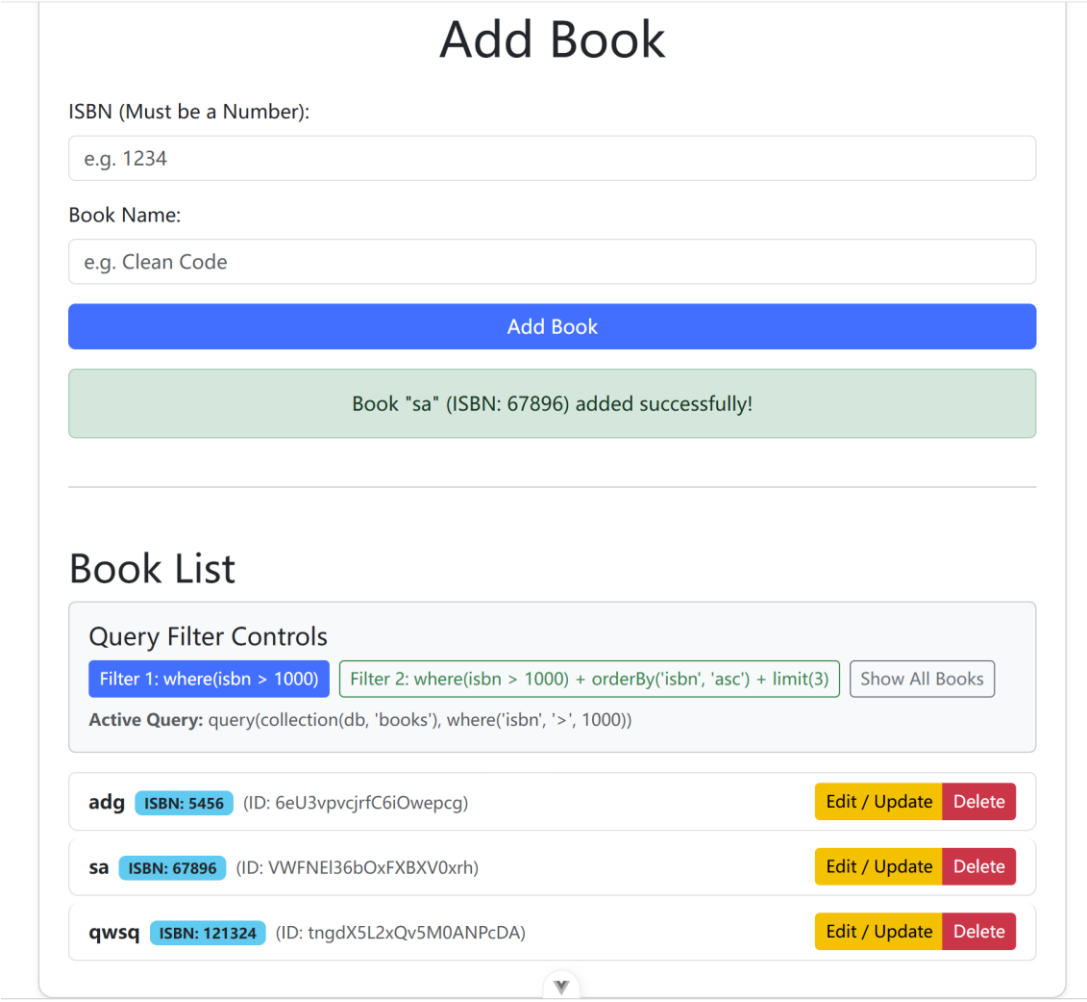





```
186  const promptUpdateBook = (book: Book) => {
187    editingBook.value = book
188    editName.value = book.name
189  }
190
191  const cancelEdit = () => {
192    editingBook.value = null
193    editName.value = ''
194  }
195
196  const confirmUpdateBook = async () => {
197    if (!editingBook.value || !editName.value.trim()) return
198    try {
199      const bookDocRef = doc(db, 'books', editingBook.value.id)
200      await updateDoc(bookDocRef, {
201        name: editName.value
202      })
203      console.log(`Book ${editingBook.value.id} updated successfully to:`, editName.value)
204      alert(`Book updated successfully to "${editName.value}"!`)
205      cancelEdit()
206      // Refresh current query
207      if (activeFilter.value === 'advanced') {
208        await fetchBooksAdvanced()
209      } else {
210        await fetchBooksWhere()
211      }
212    } catch (error) {
213      console.error('Error updating book:', error)
214    }
215  }
216
```

```
218   const deleteBook = async (bookId: string, bookTitle: string) => {
219     if (!confirm(`Are you sure you want to delete "${bookTitle}"?`)) return
220     try {
221       const bookDocRef = doc(db, 'books', bookId)
222       await deleteDoc(bookDocRef)
223       console.log(`Book ${bookId} deleted successfully!`)
224       alert(`Book "${bookTitle}" deleted successfully!`)
225       // Refresh current query
226       if (activeFilter.value === 'advanced') {
227         await fetchBooksAdvanced()
228       } else {
229         await fetchBooksWhere()
230       }
231     } catch (error) {
232       console.error('Error deleting book:', error)
233     }
234   }
```

Screenshot set 2: Screenshot of browser and visual studio code that indicates you can retrieve the database using where, orderby and limit, there are no restrictions of how many queries you need to use to do so.



```
const fetchBooksAdvanced = async () => {
  loading.value = true
  activeFilter.value = 'advanced'
  currentQueryDescription.value =
    "query(collection(db, 'books'), where('isbn', '>', 1000), orderBy('isbn', 'asc'), limit(3))"
  try {
    const q = query(
      collection(db, 'books'),
      where('isbn', '>', 1000),
      orderBy('isbn', 'asc'),
      limit(3)
    )
    const querySnapshot = await getDocs(q)
    const booksArray: Book[] = []
    querySnapshot.forEach((docSnap) => {
      booksArray.push({ id: docSnap.id, ...(docSnap.data() as { isbn: number; name: string }) })
    })
    books.value = booksArray
  } catch (error) {
    console.error('Error fetching books with where+orderBy+limit:', error)
  } finally {
    loading.value = false
  }
}
```

Github